

Static view: Utilities

Application Layer

oradio_control.py
<<module>>

+ execute(): void

Middle-ware Layer

system_sounds.py
<<module>>

+ play_sound()

error_service.py
<<module>>

+ xxxx
+ xxxx

oradio_logging.py
<<module>>

+ oradio_log()

<<uses>>

throttled_monitor.py
<<module>>

- throttled_monitor: RpiThrottlingMonitor()

<<uses>>

<<uses>>

throttled_monitor.RpiThrottlingMonitor
<<class>>

- init()
+ start()
+ stop()
+ force_throttled()
+ clear_throttled()

oradio_log:
Implementation detail

<<module>>
messaging.py

+ put_error_message(message: ErrorMessage)

Hardware Abstraction Layer

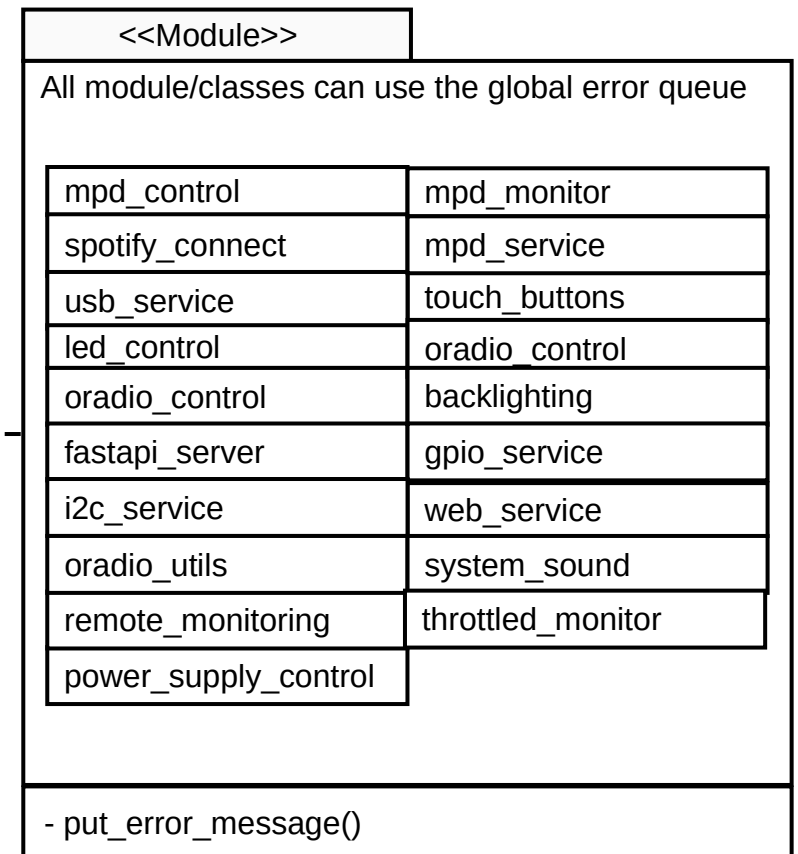
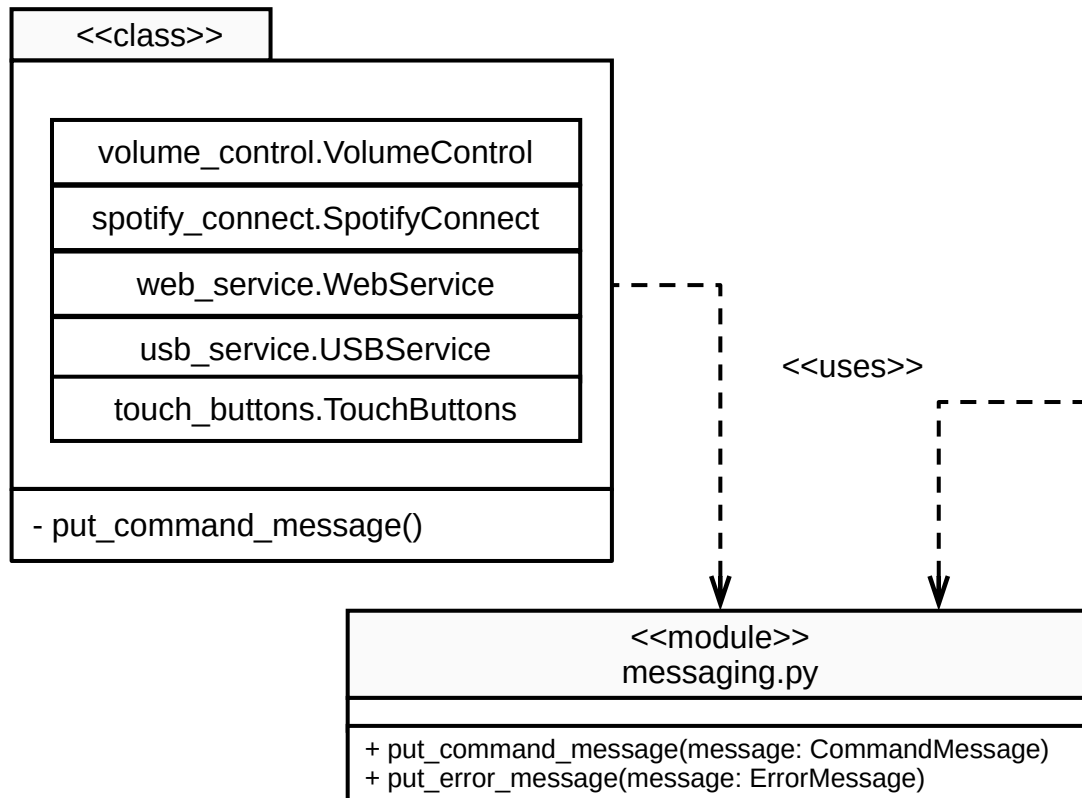
Static view: Messages

Application Layer

oradio_control.py
<<module>>

+ execute(): void

Middle-ware Layer



Hardware Abstraction Layer

Static view: Remote monitoring

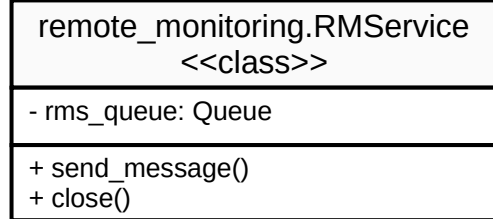
Application Layer

oradio_control.py
<<module>>

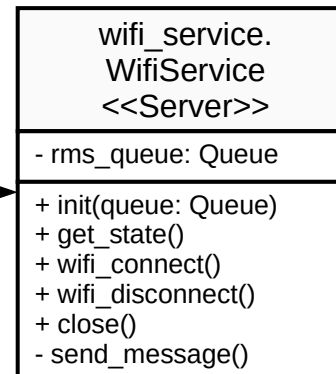
+ execute(): void

Middle-ware Layer

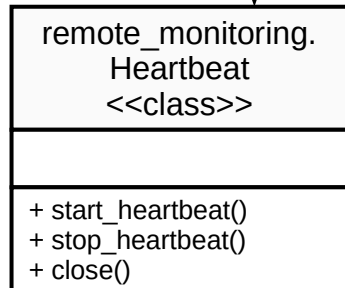
<<uses>>



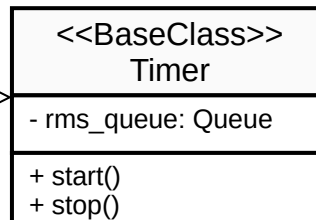
<<uses>>



<<uses>>



<<is a>>



Hardware Abstraction Layer

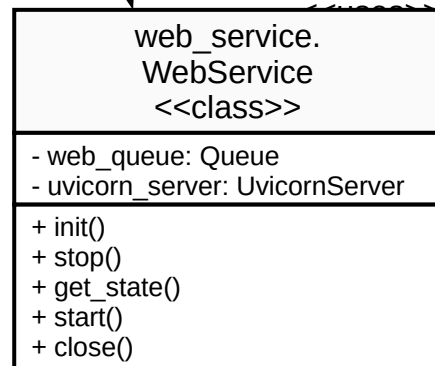
Static view: Web Service

Application Layer

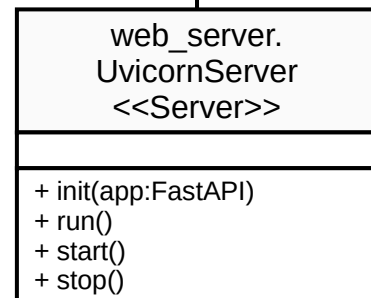
oradio_control.py
<<module>>

+ execute(): void

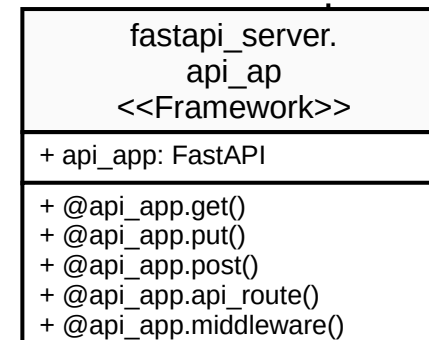
Middle-ware Layer



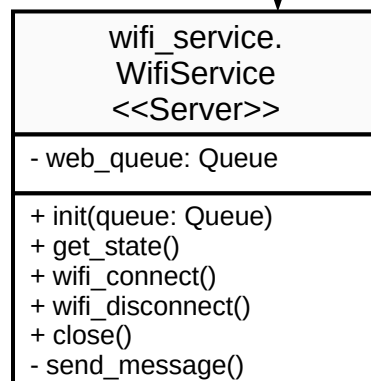
<<consists of>>



<<uses>>



<<uses>>



Hardware Abstraction Layer

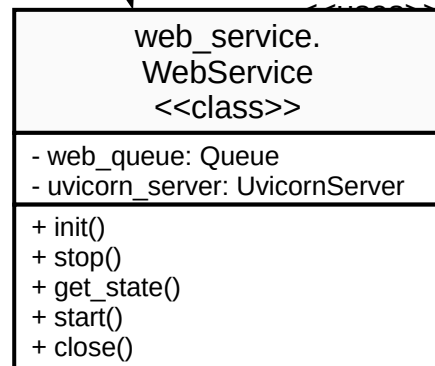
Static view: WiFi Service

Application Layer

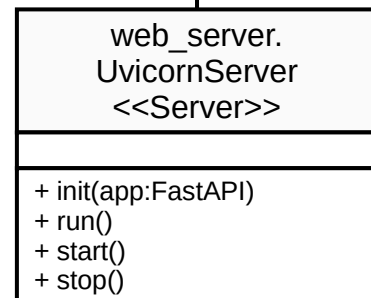
oradio_control.py
<<module>>

+ execute(): void

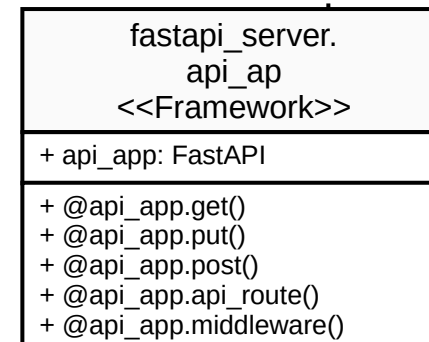
Middle-ware Layer



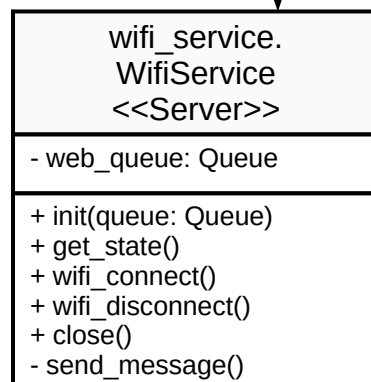
<<consists of>>



<<uses>>



<<uses>>



Hardware Abstraction Layer

Static view: USB

Application Layer

oradio_control.py
<<module>>

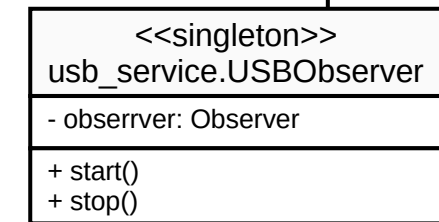
+ execute(): void

Middle-ware Layer

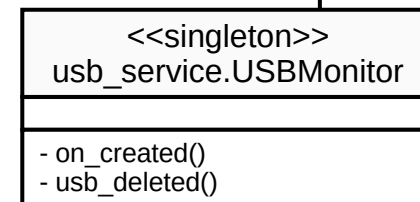
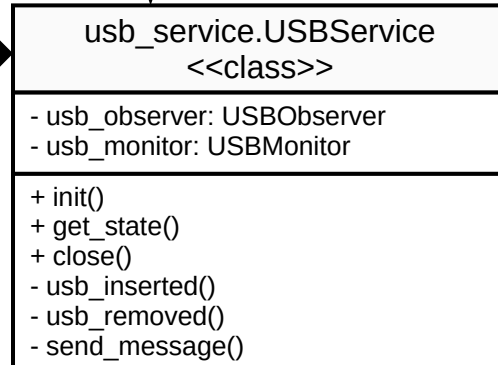
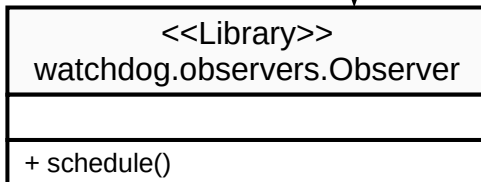
<<uses>>

<<consists of>>

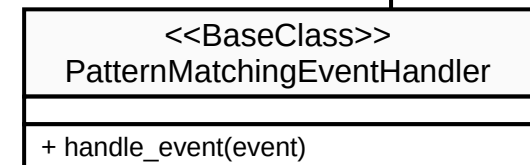
<<consists of>>



<<uses>>



<<is a>>



Hardware Abstraction Layer

Static view: Volume

Application Layer

oradio_control.py
<<module>>

+ execute(): void

Middle-ware Layer

<<uses>>

Waarom start(), stop(), set_notify().
Staat altijd op notify. Dus notify default
aan. Gaat nooit gestopt worden

volume_control.VolumeControl
<<class>>

+ set_notify()
+ start()
+ stop()

<<has>>

i2c_service.I2CService
<<class>>

+ read_byte(device: int, register: int)
+ write_byte(device: int, register: int)
+ read_block(device: int, register: int, length: int)
+ write_block(device: int, register: int, length: int)

Hardware Abstraction Layer

Static view: Error

Application Layer

oradio_control.py
<<module>>

+ execute(): void

Middle-ware Layer

<<uses>>

error_service.py.ErrorService
<<class>>

- init()
- error_handler()

```
from messaging import (  
    ErrorMessage,  
    get_error_message,  
    put_error_message,  
    CommandMessage,  
    get_command_message,  
    put_command_message,  
)
```

<<uses>>

<<module>>
messaging.py

+ ErrorMessage

+ get_error_message(): ErrorMessage

Hardware Abstraction Layer

Static view: Messaging

Application Layer

oradio_control.py
<<module>>

+ execute(): void

Middle-ware Layer

<<uses>>

messaging
<<module>>

- command_queue: Queue
- error_queue: Queue

+ put_command_message(message: CommandMessage)
+ get_command_message()
+ put_error_message(message: ErrorMessage)
+ get_error_message()

messaging.CommandMessage
<<data>>

+ source: String
+ message: String

+ is_valid()

messaging.ErrorMessage
<<data>>

+ source: String
+ message: String

+ is_valid()

Hardware Abstraction Layer

Commandmessage:
+ source: string
+ command: string
+ data: Optional[List[Any]]

Errormessage:
+ source: string
+ message: string
+ data: Optional[List[Any]]
Message: Error | Exception | ...

Static view: Power Supply

Application Layer

oradio_control.py
<<module>>

+ execute(): void

Middle-ware Layer

<<uses>>

power_supply_control.PowerSupplyService
<<class>>

+ set_nom_voltage()
+ set_max_voltage()
+ set_standby_voltage()
+ read_status()

<<has>>

<<singleton>>

i2c_service.I2CService

+ read_byte(device: int, register: int)
+ write_byte(device: int, register: int)
+ read_block(device: int, register: int, length: int)
+ write_block(device: int, register: int, length: int)

Hardware Abstraction Layer

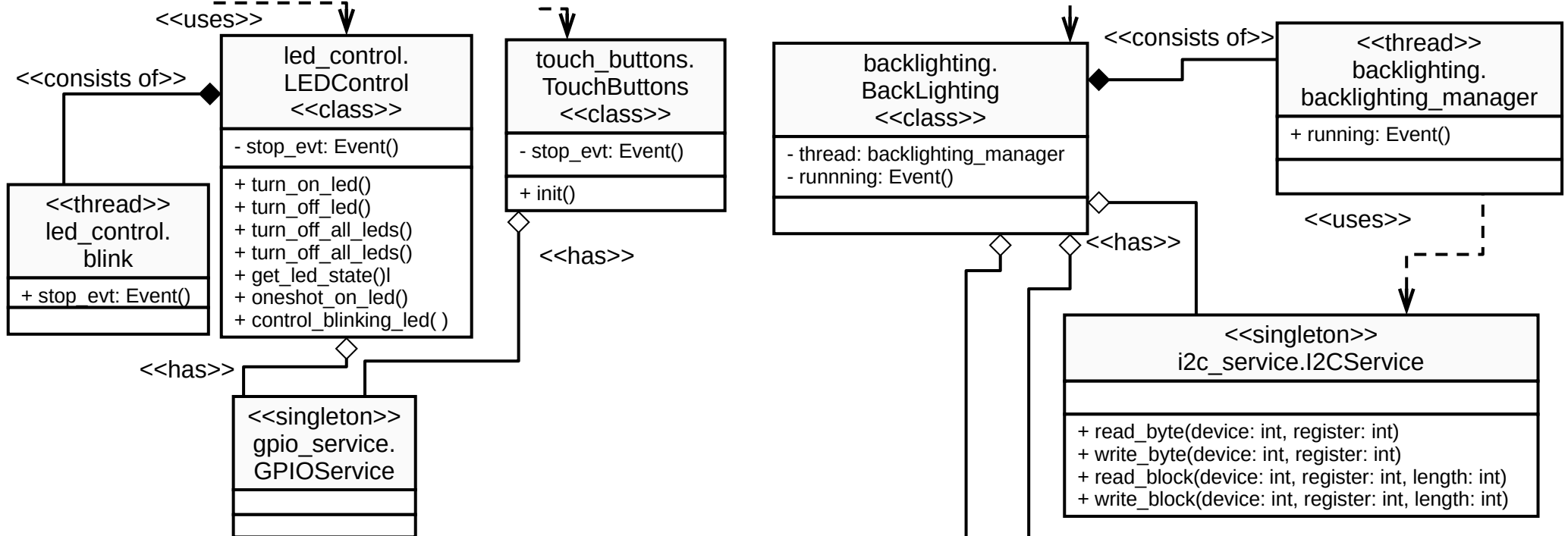
static view: Front-panel

Application Layer

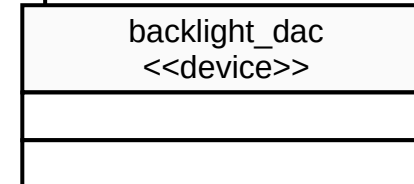
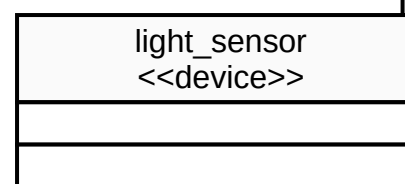
oradio_control.py
<<module>>

+ execute(): void

Middle-ware Layer



Hardware Abstraction Layer



Waarom backlighting thread met stop en off. Staat eigenlijk altijd aan! Wordt ook in de module gestart!

Static View: Music Players

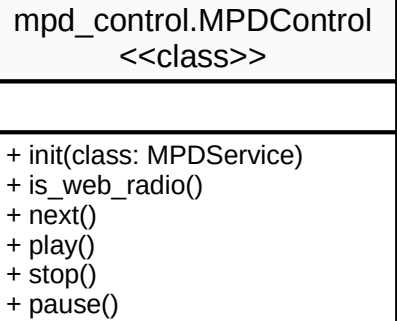
Application Layer

oradio_control.py
<<module>>

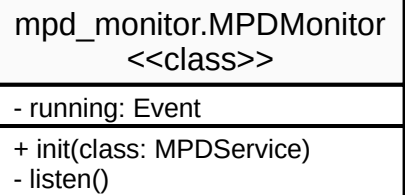
+ execute(): void

Middle-ware Layer

<<uses>>



<<uses>>



<<thread>>
mpd_monitor.
listen

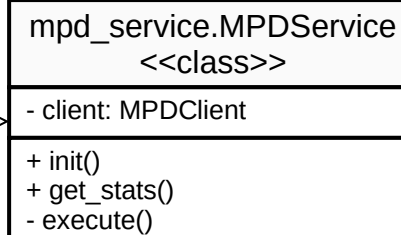
+ running: Event

<<uses>>

spotify_connect.SpotifyConnect
<<class>>

+ play()
+ pause()
+ get_state()

<<is A>>



mpd.MPDClient
<<library>>

+ init()
+ idle()
+ update()
+ next()
+ play()
+ stop()
+ pause()
+ status()

<<consists of>>

<<is A>>

Hardware Abstraction Layer